VulnLens

Smart Contract Security Analysis Report

Contract Information

Contract	safe.sol
Solidity Version	^0.8.20
Source Lines	21
Generated	17 October 2026, 03:46

Executive Summary

VulnLens identified **1** finding(s) in the submitted Solidity contract. Findings are ranked from High to Medium to Low to Informational. Each finding contains static evidence, relevant source context, contextual reasoning, and remediation guidance.

High	Medium	Low	Informational
0	**0**	**0**	**1**

Ranked Security Findings

Finding 1 — Informational

Detector	solc-version
Affected Function	Not available
Detected Source Lines	3
Static Confidence	High

Static Analysis Evidence

• VerbatimInvalidDeduplication

• FullInlinerNonExpressionSplitArgumentEvaluationOrder

• MissingSideEffectsOnSelectorAccess.

• ^0.8.20 (safe.sol#3)

Relevant Solidity Context

```
1: // SPDX-License-Identifier: MIT
2: pragma solidity ^0.8.20;
3:
4: contract SafeContract {
5:     mapping(address => uint256) public balances;
6:
7:     function deposit() external payable {
8:         balances[msg.sender] += msg.value;
```

Contextual LLM Assessment

The detector flags pragma solidity ^0.8.20 in safe.sol#3 as using a Solidity compiler version with known severe issues (VerbatimInvalidDeduplication, FullInlinerNonExpressionSplitArgumentEvaluationOrder, MissingSideEffectsOnSelectorAccess). This is an informational detector warning about the toolchain, not a confirmed vulnerability in the contract logic. No affected function is identified, and the evidence does not demonstrate that any specific contract code triggers these compiler bugs.

Exploitability: Conditional and theoretical. Exploitability depends on whether the compiled contract code patterns trigger one of the listed Solidity compiler bugs; the supplied evidence does not confirm that SafeContract's code is affected or that an exploit path exists.

Attack Scenario: If the contract code triggers VerbatimInvalidDeduplication, FullInlinerNonExpressionSplitArgumentEvaluationOrder, or MissingSideEffectsOnSelectorAccess during compilation, the deployed bytecode could behave differently than intended by the source code, potentially causing incorrect state updates or access control issues. However, the evidence is insufficient to determine whether SafeContract's deposit() or other code triggers these bugs.

Remediation Steps

1. Upgrade the pragma to a Solidity compiler version newer than 0.8.20 that resolves VerbatimInvalidDeduplication, FullInlinerNonExpressionSplitArgumentEvaluationOrder, and MissingSideEffectsOnSelectorAccess (for example, ^0.8.21 or later as advised by the Solidity bugs page)
2. Review the Solidity known bugs page and recompile/test the contract with a fixed compiler version to verify deployed bytecode behavior matches source intent

Analysis Methodology

• The Solidity source file is validated before analysis.

• Slither performs static analysis on the submitted contract.

• Detector results are converted into structured security evidence.

• Relevant Solidity source context is selected around reported locations.

• High, medium, and low impact findings are supplied to the configured LLM for contextual reasoning.

• Findings are ranked from High to Medium to Low to Informational.

• The final report combines static-analysis evidence, source context, contextual reasoning, and remediation guidance.

Scope and Limitations

• VulnLens is a pre-deployment smart-contract analysis system.

• The system does not deploy contracts.

• The system does not interact with a blockchain or ledger.

• The system does not use wallets or private keys.

• Static-analysis findings identify suspicious or security-relevant patterns and require contextual review.

• LLM output is contextual reasoning over supplied evidence and source code and should not replace independent security review.

• The absence of a finding does not prove that a contract is completely secure.

Generated by VulnLens